

```
from database import db
from sqlalchemy.orm import Mapped, mapped_column,
relationship
from sqlalchemy import String, Integer, ForeignKey
from course import Course

@property
def remaning_seats(self) -> int:
    return self.seats - len(self.participants)

def to_dict(self) -> dict:
    Afficher plus ▾
```

Voici une version corrigée de ton modèle `Participant`, avec les erreurs corrigées et des commentaires explicatifs.

</> Python



▶ Exécuter

```
from database import db
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import String, Integer, ForeignKey
from course import Course

class Participant(db.Model):
    """
    Représente un participant inscrit à une course.

    Un participant possède des informations personnelles et est lié
    à une seule course via une clé étrangère.
    """

    __tablename__ = "participants"

    # Identifiant unique du participant
    id: Mapped[int] = mapped_column(primary_key=True)

    # Nom de famille du participant
    lastname: Mapped[str] = mapped_column(String(100), nullable=False)

    # Prénom du participant
    firstname: Mapped[str] = mapped_column(String(100), nullable=False)
```

```
# Âge du participant
age: Mapped[int] = mapped_column(Integer, nullable=False)

# Prix payé par le participant en centimes
# Exemple : 2500 = 25,00 €
price: Mapped[int] = mapped_column(Integer, nullable=False)

# Clé étrangère vers la table courses
course_id: Mapped[int] = mapped_column(
    ForeignKey("courses.id"),
    nullable=False
)

# Relation avec la course associée
# Une course peut avoir plusieurs participants
course: Mapped["Course"] = relationship(
    back_populates="participants"
)

@property
def remaining_seats(self) -> int:
    """
    Retourne le nombre de places restantes dans la course.

    Calcul :
    Nombre total de places - nombre de participants inscrits
    """
    return self.course.seats - len(self.course.participants)

def to_dict(self) -> dict:
    """
    Transforme un participant en dictionnaire.

    Utile pour renvoyer les données au format JSON dans une API.
    Le prix est converti de centimes vers euros.
    """
    return {
        "id": self.id,
        "lastname": self.lastname,
        "firstname": self.firstname,
        "age": self.age,
        "course_id": self.course_id,
        "price": self.price / 100
    }

def from_body(self, body: dict):
    """
    Remplit l'objet Participant à partir des données reçues
    dans une requête (par exemple un formulaire ou une API).

    Le prix reçu en euros est converti en centimes avant stockage.
    """
```

```
self.lastname = body["lastname"]
self.firstname = body["firstname"]
self.age = body["age"]

# Conversion euros -> centimes
# Exemple : 25 devient 2500
self.price = body["price"] * 100

return self
```

Corrections effectuées :

-  remaning_seats → remaining_seats (faute d'orthographe)
-  Les fonctions @property et to_dict() étaient placées **hors de la classe**, elles sont maintenant dans Participant .
-  lastname_name → lastname (erreur de nom de variable).
-  price_paid → price (l'attribut n'existait pas).
-  self.seats → self.course.seats car les places appartiennent à la course, pas au participant.
-  Ajout de nullable=False sur course_id pour éviter un participant sans course.
-  Ajout de commentaires et de docstrings pour expliquer chaque partie.
-  Correction de la conversion du prix en centimes.

Petite remarque : la propriété remaining_seats serait généralement mieux placée dans le modèle Course , car c'est la course qui possède les places et la liste des participants. Dans Participant , elle oblige à passer par self.course .

